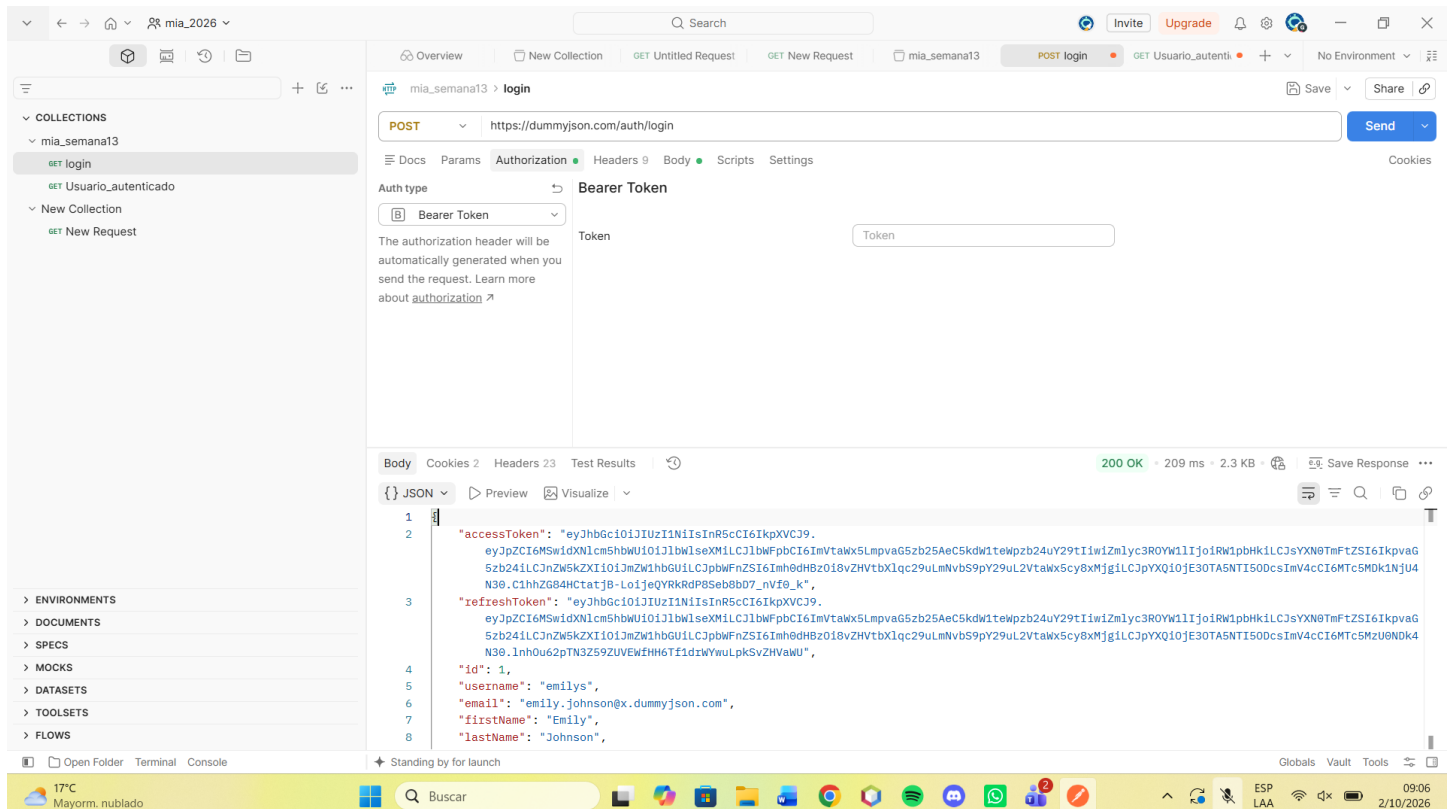






2. Utilizar token

- Cree una segunda solicitud llamada: Usuario autenticado.
- Seleccione GET.
- Utilice: <https://dummyjson.com/user/me>
- En Authorization seleccione Bearer Token.
- Copie el accessToken obtenido en la solicitud Login.
- Ejecute la solicitud.
- Verifique que la API devuelve la información del usuario autenticado.

The screenshot shows the Postman interface with a collection named 'mia_semana13'. A request named 'Usuario autenticado' is selected, using the GET method on the URL 'https://dummyjson.com/user/me'. The authorization is set to 'Bearer Token', and the token field contains a masked access token. The response is a 200 OK status with a JSON body containing user information.

Request Details:

- Method: GET
- URL: <https://dummyjson.com/user/me>
- Auth type: Bearer Token
- Token: [Masked Access Token]

Response Body (JSON):

```
1 {
2   "id": 1,
3   "firstName": "Emily",
4   "lastName": "Johnson",
5   "maidenName": "Smith",
6   "age": 29,
7   "gender": "female",
8   "email": "emily.johnson@x.dummyjson.com",
9   "phone": "+81 965-431-3824",
10  "username": "emilys",
11  "password": "emilypass",
12  "birthDate": "1996-5-38",
13  "image": "https://dummyjson.com/icon/emilys/128",
14  "bloodGroup": "O-",
15 }
```



3. Comprobar qué ocurre sin autenticación

- Duplique la solicitud Usuario autenticado.
- Quite el token.
- Ejecute nuevamente la solicitud.
- Compare la respuesta con la solicitud autenticada.
- Registre el código HTTP y explique la diferencia.

The screenshot shows a REST client interface with a sidebar on the left containing a tree view of collections and environments. The main panel displays a request configuration for a GET request to `https://dummyjson.com/user/me`. The 'Authorization' tab is active, showing 'Bearer Token' as the auth type. Below this, a text box for the token is visible. The 'Body' tab at the bottom shows a JSON response with a status of '401 Unauthorized'. The response body is:

```
{  "message": "Access Token is required"}
```

. The status bar at the bottom indicates the response time as 250 ms and the size as 1.09 KB. The system tray at the very bottom shows the date as 2/10/2026 and the time as 09:10.

Cuando no se tiene el token de acceso, no se puede realizar ninguna actividad ya que este llave de acceso sirve para ingresar a los datos sin ningún usuario y contraseña.

4. Evidencia

- Captura de Login mostrando la respuesta.
- Captura de la solicitud autenticada.
- Captura de la prueba sin token.
- Indique qué función cumple el Bearer Token.

La documentación de DummyJSON muestra el flujo de login, la obtención del accessToken y su envío mediante el encabezado Authorization: Bearer.



Parte 2 – Proteger la Connection String de Azure Storage

Utilice como base el programa C# desarrollado anteriormente para conectarse a Azure Storage. No se requiere crear nuevamente el Storage Account

5. Localizar la Connection String

- Abra el proyecto C# de Azure Storage.
- En el Explorador de soluciones, abra el archivo donde actualmente está escrita la Connection String. En un proyecto sencillo normalmente será Program.cs.
- Busque la variable que contiene la Connection String.
- Identifique el valor que comienza con información como DefaultEndpointsProtocol=...

6. Reemplazar la Connection String

Reemplace el valor escrito directamente en el programa por una lectura de una variable de entorno llamada: **AZURE_STORAGE_CONNECTION_STRING**

La idea es que el programa siga utilizando la misma variable para conectarse a Azure, pero ahora el valor será obtenido desde el entorno y no desde el código

7. Crear la variable para ejecutar localmente

- En la carpeta principal del proyecto, cree un archivo llamado .env.
- Dentro del archivo agregue la variable:
AZURE_STORAGE_CONNECTION_STRING=TU_CONNECTION_STRING
- Reemplace SU_CONNECTION_STRING por la Connection String de su Storage Account.
- Verifique que el programa lea la variable desde el .env y ejecute nuevamente el programa.
- Compruebe que el programa continúa funcionando correctamente.
- No publique el archivo .env en GitHub**, ya que contiene la credencial real.

Programa funcionando con el .env agregado

The screenshot shows the Visual Studio Code interface. The Explorer pane on the left shows the project structure with files like bin, obj, .env, BlobStorageService.cs, MenuPrincipal.cs, MIA_AzureBlob.csproj, and README.md. The .env file is open in the editor, showing the variable AZURE_STORAGE_CONNECTION_STRING. The Output pane at the bottom shows the program's execution, displaying a menu with options: 1. Subir archivo, 2. Listar archivos, 3. Descargar archivo, 4. Eliminar archivo, 5. Salir. The program is waiting for a selection.



8. Crear el Secret en Github

- Ingrese al repositorio de su proyecto en GitHub.
- Seleccione Settings.
- En el menú lateral seleccione Secrets and variables.
- Seleccione Actions.
- Seleccione New repository secret.
- En Name escriba: AZURE_STORAGE_CONNECTION_STRING
- En Secret coloque la Connection String de Azure Storage.
- Seleccione Add secret.

Creación del secret en github

The screenshot shows the GitHub web interface for the repository 'gmperezgom/url-mia-2026'. The left sidebar contains a navigation menu with categories like 'Code, planning, and automation' and 'Security and quality'. The 'Secrets and variables' section is expanded, and 'Actions' is selected. The main content area shows the 'Secrets' tab for 'Environment secrets' and 'Repository secrets'. The 'Repository secrets' section has a 'New repository secret' button and a table with one secret: 'AZURE_STORAGE_CONNECTION_STRING'.

Name	Last updated
AZURE_STORAGE_CONNECTION_STRING	now

Entrega

Dentro del repositorio de GitHub, cree la carpeta semana-13/lab2/ y coloque en ella un PDF con la solución del laboratorio.

Incluya también los archivos de respaldo generados durante la práctica, así como los pantallazos que evidencien las pruebas realizadas.

Enviar link de github a plataforma de la universidad.